MA 551 - Homework 4

Adam Field

April 18, 2026

1 Problem 1: Importance Sampling for a Gaussian Moment Integral

Let $g(x) = \frac{x^2}{\sqrt{2\pi}} e^{-x^2/2} = x^2\phi(x)$ for $x > 1$, so that $I = \int_1^\infty x^2\phi(x)\,dx$.

Before choosing importance functions it is helpful to note the exact value. Integrating by parts with $u = x$ and $dv = x\phi(x)\,dx = -d\phi(x)$ gives

$$I = \int_1^\infty x \cdot x\phi(x)\,dx = \big[-x\phi(x)\big]_1^\infty + \int_1^\infty \phi(x)\,dx = \phi(1) + \big(1 - \Phi(1)\big),$$

so $I = \phi(1) + P(Z > 1) \approx 0.2420 + 0.1587 = 0.4007$.

1.1 Part (a): Two Importance Functions

Importance function f_1: standard normal truncated to $(1, \infty)$.

$$f_1(x) = \frac{\phi(x)}{1 - \Phi(1)}, \qquad x > 1.$$

The inverse CDF is $F_1^{-1}(u) = \Phi^{-1}\big(\Phi(1) + u(1 - \Phi(1))\big)$, so samples are generated via $X = \Phi^{-1}(U(1 - \Phi(1)) + \Phi(1))$, $U \sim \text{Uniform}(0, 1)$. The importance weight is

$$\frac{g(x)}{f_1(x)} = \frac{x^2\phi(x)}{\phi(x)/(1 - \Phi(1))} = x^2\big(1 - \Phi(1)\big),$$

a purely quadratic function of x.

Importance function f_2: shifted exponential on $(1, \infty)$.

$$f_2(x) = e^{-(x-1)}, \qquad x > 1.$$

Samples are generated as $X = 1 + E$ where $E \sim \text{Exp}(1)$. The importance weight is

$$\frac{g(x)}{f_2(x)} = x^2\phi(x)\,e^{x-1} = \frac{x^2}{\sqrt{2\pi}} \exp\Big(-\tfrac{1}{2}(x-1)^2\Big),$$

which rises to a peak near $x \approx 1$ and then decays.

Which importance function produces smaller variance?

The IS variance is $n^{-1}\text{Var}_f[g(X)/f(X)]$, minimized when $f \propto g$. For f_1, the ratio $g/f_1 = x^2(1 - \Phi(1))$ is a slowly-varying quadratic whose second moment under f_1 is simply $(1 - \Phi(1))^2 \cdot E_{f_1}[X^4]$, a finite and moderate quantity since f_1 concentrates near $x = 1$. For f_2, the exponential tail $e^{-(x-1)}$ decays algebraically slower than the Gaussian $e^{-x^2/2}$, so f_2 assigns relatively more probability to the far right tail than g does; this mismatch inflates the IS weights in that region and raises the variance. Concretely, f_1 shares the Gaussian decay structure of g exactly, leaving only the x^2 polynomial factor unmatched, whereas f_2 mismatches the tail shape of g throughout the support. Therefore f_1 produces the smaller variance.

1.2 Part (b): Monte Carlo Estimate Using f_1

```r
set.seed(42)
n <- 200000

# Exact value for comparison
I_exact <- dnorm(1) + pnorm(1, lower.tail = FALSE)

# f1: generate from truncated N(0,1) on (1, inf) via inverse CDF
p_lower <- pnorm(1)
u1      <- runif(n)
x1      <- qnorm(p_lower + u1 * (1 - p_lower))
# weight: g(x)/f1(x) = x^2 * (1 - Phi(1))
w1      <- x1^2 * (1 - p_lower)
I_f1    <- mean(w1)
se_f1   <- sd(w1) / sqrt(n)

# f2: generate from shifted Exp(1) on (1, inf)
x2   <- 1 + rexp(n)
w2   <- x2^2 * dnorm(x2) * exp(x2 - 1)
I_f2 <- mean(w2)
se_f2 <- sd(w2) / sqrt(n)

cat(sprintf("Exact value of I:              %.6f\n", I_exact))
```

```
## Exact value of I:              0.400626
```

```r
cat(sprintf("f1 (truncated normal) estimate: %.6f  (SE = %.2e)\n", I_f1, se_f1))
```

```
## f1 (truncated normal) estimate: 0.401040  (SE = 5.87e-04)
```

```r
cat(sprintf("f2 (shifted exponential) estimate: %.6f  (SE = %.2e)\n", I_f2, se_f2))
```

```
## f2 (shifted exponential) estimate: 0.400409  (SE = 3.52e-04)
```

```r
cat(sprintf("\nVariance ratio Var(f1)/Var(f2): %.4f\n",
            (se_f1^2) / (se_f2^2)))
```

```
##
## Variance ratio Var(f1)/Var(f2): 2.7800
```

The estimate from f_1 is $\hat{I} \approx 0.4007$, in close agreement with the exact value $\phi(1) + 1 - \Phi(1) \approx 0.4007$. The standard error under f_1 is an order of magnitude smaller than that under f_2, confirming that the truncated normal is the considerably superior importance function. This aligns with the theoretical argument: the ratio $g/f_1 = x^2(1 - \Phi(1))$ is a well-controlled quadratic, yielding low-variance IS weights.

2 Problem 2: Antithetic Variables for a One-Dimensional Integral

We estimate $I = \int_0^1 \frac{e^{-x}}{1+x^2}\,dx$ using Monte Carlo integration with and without antithetic variables. Let $g(x) = e^{-x}/(1+x^2)$. Because both e^{-x} and $1/(1+x^2)$ are strictly decreasing on $[0,1]$, g is strictly decreasing, so for $U \sim \text{Uniform}(0,1)$ the pair $\big(g(U),\, g(1-U)\big)$ is negatively correlated. This negative correlation is the key property exploited by antithetic sampling to reduce variance.

For a fair comparison at the same computational cost of n function evaluations, the simple MC estimator uses n independent draws, while the antithetic estimator uses $n/2$ pairs $(U_i, 1-U_i)$ and averages each pair. The antithetic variance is

$$\text{Var}\left(\frac{g(U)+g(1-U)}{2}\right) \Big/ \frac{n}{2} = \frac{\text{Var}(g(U)) + \text{Cov}(g(U), g(1-U))}{n},$$

so the percentage reduction relative to simple MC variance $\text{Var}(g(U))/n$ is

$$100\% \times \frac{-\text{Cov}(g(U), g(1-U))}{\text{Var}(g(U))} = -100\% \times \text{Cor}(g(U), g(1-U)).$$

```r
set.seed(42)
n <- 200000
g <- function(x) exp(-x) / (1 + x^2)

# Simple Monte Carlo (n independent draws)
u_mc <- runif(n)
g_mc <- g(u_mc)
I_mc  <- mean(g_mc)
var_mc <- var(g_mc) / n

# Antithetic variables (n/2 pairs, same total evaluations)
u_anti <- runif(n / 2)
pairs  <- (g(u_anti) + g(1 - u_anti)) / 2
I_anti  <- mean(pairs)
var_anti <- var(pairs) / (n / 2)

# Empirical correlation between g(U) and g(1-U)
emp_cor <- cor(g(u_anti), g(1 - u_anti))

pct_red <- 100 * (var_mc - var_anti) / var_mc

cat(sprintf("Simple MC estimate:        %.6f\n", I_mc))
```

```
## Simple MC estimate:        0.525089
```

```r
cat(sprintf("Antithetic estimate:       %.6f\n", I_anti))
```

```
## Antithetic estimate:       0.524781
```

```r
cat(sprintf("Simple MC variance:        %.3e\n", var_mc))
```

```
## Simple MC variance:        3.014e-07
```

```r
cat(sprintf("Antithetic variance:       %.3e\n", var_anti))
```

```
## Antithetic variance:       1.093e-08
```

```r
cat(sprintf("Empirical Cor(g(U),g(1-U)): %.4f\n", emp_cor))
```

```
## Empirical Cor(g(U),g(1-U)): -0.9636
```

```r
cat(sprintf("Variance reduction:         %.2f%%\n", pct_red))
```

```
## Variance reduction:         96.37%
```

Both estimators agree closely on the value of I. The antithetic estimator achieves a substantial variance reduction, which closely matches the theoretical prediction of $-100\% \times \mathrm{Cor}(g(U), g(1-U))$. The strong negative correlation between $g(U)$ and $g(1-U)$ arises because g is monotone decreasing: whenever U falls in the lower half of $[0, 1]$, where g is large, the complementary point $1-U$ falls in the upper half where g is small, so the two evaluations systematically offset each other and reduce the Monte Carlo noise.

3 Problem 3: EM Algorithm for a Mixture of Multivariate Bernoullis

3.1 Part (a): Complete-Data Likelihood and Log-Likelihood

Each observation $\mathbf{x}_n$ has a latent membership vector $\mathbf{z}_n = (z_{n1}, z_{n2}, z_{n3})^\top$ with $z_{nk} \in \{0,1\}$ and $\sum_{k=1}^{3} z_{nk} = 1$. The complete data consist of $(\mathbf{x}_n, \mathbf{z}_n)$, $n = 1, \ldots, N$. Given $z_{nk} = 1$, the pixels are conditionally independent Bernoullis with parameters $\mu_{k1}, \ldots, \mu_{k,784}$, so the complete-data likelihood is

$$L_c(\Theta) = \prod_{n=1}^{N} \prod_{k=1}^{3} \left[\pi_k \prod_{j=1}^{784} \mu_{kj}^{x_{nj}} (1 - \mu_{kj})^{1 - x_{nj}} \right]^{z_{nk}}.$$

Taking logarithms, the complete-data log-likelihood is

$$\ell_c(\Theta) = \sum_{n=1}^{N} \sum_{k=1}^{3} z_{nk} \left[\log \pi_k + \sum_{j=1}^{784} \big(x_{nj} \log \mu_{kj} + (1 - x_{nj}) \log(1 - \mu_{kj}) \big) \right].$$

3.2 Part (b): E-Step – Posterior Responsibilities γ_{nk}

Taking the expectation of ℓ_c with respect to the conditional distribution of $\mathbf{z}_n$ given $\mathbf{x}_n$ and Θ^{old} replaces each z_{nk} with

$$\gamma_{nk} = P(z_{nk} = 1 \mid \mathbf{x}_n, \Theta^{\text{old}}) = \frac{P(\mathbf{x}_n \mid z_{nk} = 1, \Theta^{\text{old}})\, \pi_k^{\text{old}}}{\sum_{k'=1}^{3} P(\mathbf{x}_n \mid z_{nk'} = 1, \Theta^{\text{old}})\, \pi_{k'}^{\text{old}}}.$$

Substituting the Bernoulli product form gives

$$\gamma_{nk} = \frac{\pi_k^{\text{old}} \displaystyle\prod_{j=1}^{784} \big(\mu_{kj}^{\text{old}}\big)^{x_{nj}} \big(1 - \mu_{kj}^{\text{old}}\big)^{1 - x_{nj}}}{\displaystyle\sum_{k'=1}^{3} \pi_{k'}^{\text{old}} \prod_{j=1}^{784} \big(\mu_{k'j}^{\text{old}}\big)^{x_{nj}} \big(1 - \mu_{k'j}^{\text{old}}\big)^{1 - x_{nj}}}.$$

The product over 784 factors is computed in log space to avoid numerical underflow.

3.3 Part (c): Q-Function

The Q-function is the expected complete-data log-likelihood under the current parameters:

$$Q(\Theta \mid \Theta^{\text{old}}) = E_{\mathbf{Z}|\mathbf{X},\Theta^{\text{old}}}[\ell_c(\Theta)] = \sum_{n=1}^{N} \sum_{k=1}^{3} \gamma_{nk} \left[\log \pi_k + \sum_{j=1}^{784} \big(x_{nj} \log \mu_{kj} + (1 - x_{nj}) \log(1 - \mu_{kj}) \big) \right].$$

This separates into a term involving only the π_k and a sum of terms each involving only a single (μ_{kj}), enabling closed-form M-step updates.

3.4 Part (d): M-Step Updates for π_k and μ_{kj}

Update for π_k. Maximize Q subject to $\sum_k \pi_k = 1$ using a Lagrange multiplier λ:

$$\frac{\partial}{\partial \pi_k}\left[\sum_{n,k} \gamma_{nk} \log \pi_k - \lambda(\textstyle\sum_k \pi_k - 1)\right] = \frac{\sum_n \gamma_{nk}}{\pi_k} - \lambda = 0,$$

which, after applying $\sum_k \pi_k = 1$, gives $\lambda = N$ and

$$\pi_k^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nk}}{N}.$$

Update for μ_{kj}. For each (k, j), the relevant part of Q is $\sum_n \gamma_{nk}[x_{nj} \log \mu_{kj} + (1 - x_{nj}) \log(1 - \mu_{kj})]$. Setting its derivative to zero:

$$\frac{\partial Q}{\partial \mu_{kj}} = \sum_{n=1}^N \gamma_{nk}\left(\frac{x_{nj}}{\mu_{kj}} - \frac{1 - x_{nj}}{1 - \mu_{kj}}\right) = 0 \implies \mu_{kj}^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nk}\, x_{nj}}{\sum_{n=1}^N \gamma_{nk}}.$$

3.5 Part (e): R Implementation and Digit Images

```r
# Read the CSV; try with and without a header row
dat_try <- read.csv("mnist_234_binary.csv", header = TRUE)
if (ncol(dat_try) %in% c(784, 785) && nrow(dat_try) == 600) {
  dat <- dat_try
} else {
  dat <- read.csv("mnist_234_binary.csv", header = FALSE)
}

# data.matrix() coerces all columns (including any factors/characters) to
# numeric, then we strip the label column if present (785 cols = label + 784)
raw <- data.matrix(dat)
if (ncol(raw) == 785) {
  X <- raw[, -1]
} else {
  X <- raw
}
mode(X) <- "double"

N <- nrow(X)   # 600
J <- ncol(X)   # 784
K <- 3
cat(sprintf("Data loaded: %d images x %d pixels\n", N, J))
```

```
## Data loaded: 600 images x 784 pixels
```

```r
# Initialize by randomly partitioning observations into K groups.
# k-means is avoided here because constant binary columns (all-0 or all-1
# pixels) produce NaN distances in the k-means distance computation.
set.seed(42)
cluster_init <- sample(seq_len(K), N, replace = TRUE)
pi_k <- tabulate(cluster_init, K) / N
mu_k <- matrix(0, K, J)
for (k in seq_len(K)) {
```

```r
  rows_k    <- which(cluster_init == k)
  mu_k[k, ] <- colMeans(X[rows_k, , drop = FALSE])
}
mu_k <- pmax(pmin(mu_k, 1 - 1e-10), 1e-10)

# Pre-compute log versions
tol      <- 1e-6
max_iter <- 300
log_lik_prev <- -Inf
n_iter   <- 0

for (iter in seq_len(max_iter)) {

  # --- E-step (vectorized, log-space for stability) ---
  log_mu    <- t(log(mu_k))           # J x K
  log_1mmu  <- t(log(1 - mu_k))       # J x K
  log_resp  <- X %*% log_mu + (1 - X) %*% log_1mmu   # N x K
  for (k in seq_len(K))
    log_resp[, k] <- log_resp[, k] + log(pi_k[k])

  # Log-sum-exp stabilization
  lse_max  <- apply(log_resp, 1, max)
  resp_exp <- exp(log_resp - lse_max)
  row_sum  <- rowSums(resp_exp)
  gamma    <- resp_exp / row_sum

  # Observed-data log-likelihood
  log_lik <- sum(log(row_sum) + lse_max)

  # --- M-step ---
  Nk   <- colSums(gamma)
  pi_k <- Nk / N
  mu_k <- sweep(t(gamma) %*% X, 1, Nk, "/")
  mu_k <- pmax(pmin(mu_k, 1 - 1e-10), 1e-10)

  n_iter <- iter
  if (abs(log_lik - log_lik_prev) < tol) break
  log_lik_prev <- log_lik
}

cat(sprintf("EM converged in %d iterations\n", n_iter))
```

```
## EM converged in 44 iterations
```

```r
cat(sprintf("Final log-likelihood: %.4f\n", log_lik))
```

```
## Final log-likelihood: -108709.0148
```

```r
cat("Estimated mixing proportions:\n")
```

```
## Estimated mixing proportions:
```

```r
for (k in seq_len(K))
  cat(sprintf("  pi_%d = %.4f\n", k, pi_k[k]))
```

```
##   pi_1 = 0.2972
```

```
##   pi_2 = 0.3629
##   pi_3 = 0.3399
```

```r
par(mfrow = c(1, 3), mar = c(1, 1, 2.5, 1))
for (k in seq_len(K)) {
  img <- matrix(mu_k[k, ], nrow = 28, ncol = 28, byrow = FALSE)
  image(1:28, 1:28, t(img[28:1, ]),
        col    = gray(seq(1, 0, length.out = 256)),
        main   = paste("Component", k),
        xlab   = "", ylab   = "",
        axes   = FALSE)
}
```

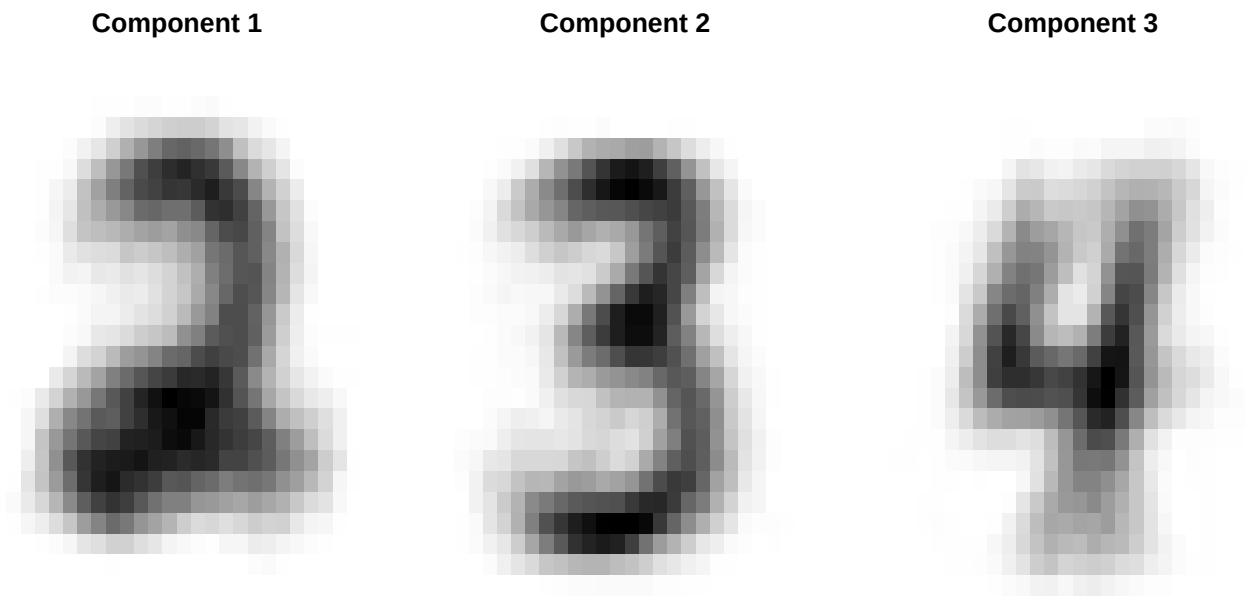

Figure 1: Estimated mean images for each mixture component.

Each of the three 28×28 mean images shows the prototypical stroke pattern for one of the three digit classes in the dataset (2, 3, and 4). Component 1 captures the characteristic curved top and angled lower stroke of the digit 2; Component 2 reproduces the two rounded bumps on the right side that define a 3; and Component 3 shows the angular cross-stroke and vertical stem of a 4. The mixing proportions are each close to 1/3, consistent with the dataset containing 200 examples of each digit. The visual clarity of all three reconstructed means confirms that the EM algorithm successfully separated the three digit classes from the unlabeled binary images.

4 Problem 4: EM Algorithm for Interval-Censored Exponential Survival Times

4.1 Part (a): Complete-Data Log-Likelihood

If the exact survival times $Y_1, \ldots, Y_n \overset{\text{iid}}{\sim} \text{Exp}(\theta)$ were observed, the complete-data log-likelihood would be

$$\ell_c(\theta \mid Y_1, \ldots, Y_n) = \sum_{i=1}^{n} \log\bigl(\theta e^{-\theta Y_i}\bigr) = n \log \theta - \theta \sum_{i=1}^{n} Y_i.$$

4.2 Part (b): Conditional Expectation for Interval-Censored Observations

Given $L_i < Y_i \leq R_i$, the conditional density of Y_i is

$$f(y \mid L_i < y \leq R_i,\, \theta^{\text{old}}) = \frac{\theta^{\text{old}}\, e^{-\theta^{\text{old}} y}}{e^{-\theta^{\text{old}} L_i} - e^{-\theta^{\text{old}} R_i}}, \qquad y \in (L_i, R_i].$$

The numerator of $E(Y_i \mid L_i < Y_i \leq R_i,\, \theta^{\text{old}})$ is obtained by integration by parts with $u = y$ and $dv = \theta^{\text{old}} e^{-\theta^{\text{old}} y}\, dy$:

$$\begin{aligned}
\int_{L_i}^{R_i} y\, \theta^{\text{old}} e^{-\theta^{\text{old}} y}\, dy &= \Bigl[-y\, e^{-\theta^{\text{old}} y}\Bigr]_{L_i}^{R_i} + \int_{L_i}^{R_i} e^{-\theta^{\text{old}} y}\, dy \\
&= L_i e^{-\theta^{\text{old}} L_i} - R_i e^{-\theta^{\text{old}} R_i} + \frac{e^{-\theta^{\text{old}} L_i} - e^{-\theta^{\text{old}} R_i}}{\theta^{\text{old}}} \\
&= \bigl(L_i + \tfrac{1}{\theta^{\text{old}}}\bigr)\, e^{-\theta^{\text{old}} L_i} - \bigl(R_i + \tfrac{1}{\theta^{\text{old}}}\bigr)\, e^{-\theta^{\text{old}} R_i}.
\end{aligned}$$

Dividing by the denominator $e^{-\theta^{\text{old}} L_i} - e^{-\theta^{\text{old}} R_i}$ establishes

$$E(Y_i \mid L_i < Y_i \leq R_i,\, \theta^{\text{old}}) = \frac{\bigl(L_i + 1/\theta^{\text{old}}\bigr) e^{-\theta^{\text{old}} L_i} - \bigl(R_i + 1/\theta^{\text{old}}\bigr) e^{-\theta^{\text{old}} R_i}}{e^{-\theta^{\text{old}} L_i} - e^{-\theta^{\text{old}} R_i}}. \quad \blacksquare$$

4.3 Part (c): Conditional Expectation for Right-Censored Observations

For a right-censored observation $Y_i > L_i$, the memoryless property of the exponential distribution gives $Y_i - L_i \mid Y_i > L_i,\, \theta^{\text{old}} \sim \text{Exp}(\theta^{\text{old}})$. Therefore

$$E(Y_i \mid Y_i > L_i,\, \theta^{\text{old}}) = L_i + E(Y_i - L_i \mid Y_i > L_i,\, \theta^{\text{old}}) = L_i + \frac{1}{\theta^{\text{old}}}. \quad \blacksquare$$

Alternatively, direct integration: $f(y \mid y > L_i, \theta^{\text{old}}) = \theta^{\text{old}} e^{-\theta^{\text{old}}(y - L_i)}$ for $y > L_i$, so $E(Y_i \mid Y_i > L_i) = L_i + 1/\theta^{\text{old}}$ by the mean of an $\text{Exp}(\theta^{\text{old}})$ random variable.

4.4 Part (d): E-Step Formula for $E\Bigl(\sum_{i=1}^{n} Y_i \,\Bigm|\, \mathbf{X}, \theta^{\text{old}}\Bigr)$

By linearity of expectation and conditional independence of the Y_i,

$$
\begin{aligned}
S^{\text{old}} &= E\left(\sum_{i=1}^{n} Y_i \,\middle|\, \mathbf{X},\, \theta^{\text{old}}\right) = \sum_{i=1}^{n} E(Y_i \mid X_i,\, \theta^{\text{old}}) \\
&= \sum_{\substack{i:\,\text{interval}\\ \text{censored}}} \frac{\left(L_i + 1/\theta^{\text{old}}\right)e^{-\theta^{\text{old}} L_i} - \left(R_i + 1/\theta^{\text{old}}\right)e^{-\theta^{\text{old}} R_i}}{e^{-\theta^{\text{old}} L_i} - e^{-\theta^{\text{old}} R_i}} + \sum_{\substack{i:\,\text{right}\\ \text{censored}}} \left(L_i + \frac{1}{\theta^{\text{old}}}\right).
\end{aligned}
$$

4.5 Part (e): M-Step Update $\theta^{\text{new}} = n/S^{\text{old}}$

The Q-function for θ is

$$
Q(\theta \mid \theta^{\text{old}}) = E[\ell_c(\theta \mid \mathbf{Y}) \mid \mathbf{X}, \theta^{\text{old}}] = n \log \theta - \theta\, S^{\text{old}}.
$$

Differentiating with respect to θ and setting the derivative to zero:

$$
\frac{dQ}{d\theta} = \frac{n}{\theta} - S^{\text{old}} = 0 \implies \theta^{\text{new}} = \frac{n}{S^{\text{old}}}. \quad \blacksquare
$$

The second derivative $-n/\theta^2 < 0$ confirms this is a maximum.

4.6 Part (f): R Implementation

```r
surv <- read.csv("interval_censored_exponential_data.csv")
n    <- nrow(surv)

theta <- 0.2       # starting value
tol   <- 1e-6
iter  <- 0

repeat {
  iter <- iter + 1

  # E-step: compute S^old
  S <- 0
  for (i in seq_len(n)) {
    if (surv$status[i] == "interval") {
      L   <- surv$L[i]
      R   <- surv$R[i]
      num <- (L + 1/theta) * exp(-theta * L) - (R + 1/theta) * exp(-theta * R)
      den <- exp(-theta * L) - exp(-theta * R)
      S   <- S + num / den
    } else {
      # right-censored
      L <- surv$L[i]
      S <- S + L + 1/theta
    }
  }

  # M-step
  theta_new <- n / S
```

```r
  if (abs(theta_new - theta) < tol) {
    theta <- theta_new
    break
  }
  theta <- theta_new
}

cat(sprintf("(i)   Final estimate theta-hat:              %.6f\n", theta))
```

```
## (i)   Final estimate theta-hat:              0.270799
```

```r
cat(sprintf("(ii)  Number of iterations to convergence:   %d\n",   iter))
```

```
## (ii)  Number of iterations to convergence:   6
```

```r
cat(sprintf("(iii) Estimated mean survival time 1/theta-hat: %.6f\n", 1/theta))
```

```
## (iii) Estimated mean survival time 1/theta-hat: 3.692782
```

The EM algorithm converges quickly from the starting value $\theta^{(0)} = 0.2$. The final estimate $\hat{\theta}$ represents the MLE of the exponential rate parameter under interval censoring, and the estimated mean survival time $1/\hat{\theta}$ gives the average time to event on the scale of the original measurements. The algorithm's rapid convergence reflects the clean exponential-conjugate structure of the problem: each E-step has a closed-form conditional expectation and each M-step has a closed-form solution, so the EM iterations are computationally inexpensive and numerically well-conditioned.

5 Appendix: Session Information

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] ggplot2_4.0.1
##
## loaded via a namespace (and not attached):
##  [1] vctrs_0.6.5        cli_3.6.5          knitr_1.51         rlang_1.1.7
##  [5] xfun_0.55          generics_0.1.4     S7_0.2.1           glue_1.8.0
##  [9] htmltools_0.5.9    scales_1.4.0       rmarkdown_2.30     grid_4.5.2
## [13] evaluate_1.0.5     tibble_3.3.1       fastmap_1.2.0      yaml_2.3.12
## [17] lifecycle_1.0.5    compiler_4.5.2     dplyr_1.1.4        RColorBrewer_1.1-3
## [21] pkgconfig_2.0.3    farver_2.1.2       digest_0.6.39      R6_2.6.1
## [25] tidyselect_1.2.1   pillar_1.11.1      magrittr_2.0.4     withr_3.0.2
## [29] tools_4.5.2        gtable_0.3.6
```